



ŠOLSKI CENTER VELENJE  
ELEKTRO IN RAČUNALNIŠKA ŠOLA  
Trg mladosti 3  
3320 VELENJE

IZDELEK OZ. STORITEV PO IZOBRAŽEVALNEM PROGRAMU  
SREDNJEGA STROKOVNEGA IZOBRAŽEVANJA  
TEHNIK RAČUNALNIŠTVA

**IZDELAVA NAMIZNE APLIACIJE ZA BELEŽENJE  
ZAPOSLENIH V PODJETJU**

Avtor: Blaž KRISTAN

Mentor na šoli: Islam Mušić, inž

Velenje, datum



---

KAZALO VSEBINE:

|          |                                                                                                   |           |
|----------|---------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>UVOD .....</b>                                                                                 | <b>1</b>  |
| <b>2</b> | <b>PREGLED STANJA TEHNIKE .....</b>                                                               | <b>2</b>  |
| 2.1      | POSTGRESQL.....                                                                                   | 2         |
| 2.2      | DATAGRIP .....                                                                                    | 2         |
| 2.3      | SUPABASE .....                                                                                    | 3         |
| 2.4      | GITHUB.....                                                                                       | 3         |
| 2.5      | TOAD DATA MODELER .....                                                                           | 4         |
| 2.6      | INTELLIJ IDEA .....                                                                               | 5         |
| <b>3</b> | <b>METODE DELA .....</b>                                                                          | <b>6</b>  |
| 3.1      | PRIPRAVA GITHUB REPOZITORIJA.....                                                                 | 6         |
| 3.1.1    | <i>Priprava Github desktop .....</i>                                                              | <i>6</i>  |
| 3.2      | NAČRTOVANJE PODATKOVNEGA MODELA (ER-DIAGRAM) .....                                                | 7         |
| 3.3      | IZDELAVA APLIKACIJE V INTELJ.....                                                                 | 8         |
| 3.3.1    | <i>STROKTURA PROJEKTA.....</i>                                                                    | <i>8</i>  |
| 3.3.2    | <i>POTEK IZDELAVE PROJEKTA .....</i>                                                              | <i>8</i>  |
| 3.4      | ZAČETNO TESTIRANJE NA LOKALNEM OKOLJU Z DATAGRIPOM .....                                          | 9         |
| 3.5      | VZPOSTAVITEV POVEZAVE NA SUPABASE .....                                                           | 9         |
| 3.6      | TESTIRANJE DELOVANJA APLIKACIJE .....                                                             | 10        |
| <b>4</b> | <b>PROŽILCI .....</b>                                                                             | <b>11</b> |
| 4.1      | PROŽILEC 1 .....                                                                                  | 11        |
|          | SPROŽILEC ZA BELEŽENJE ČASA ZADNJE SPREMEMBE ZAPISA - TRG_LOG_ZAPOSLENI .....                     | 11        |
| 4.2      | PROŽILEC 2 .....                                                                                  | 12        |
|          | SPROŽILEC ZA PREPREČEVANJE NEVELJAVNIH VNOSOV - FIX_NEGATIVE_PLACA .....                          | 12        |
| 4.3      | PROŽILEC 3 .....                                                                                  | 13        |
|          | SPROŽILEC ZA SAMODEJNO OBDELAVO BRISANJA POVEZANIH PODATKOV - FIX_FUTURE_DATUM_ZAPOSLOTITVE ..... | 13        |
| <b>5</b> | <b>REZULTATI .....</b>                                                                            | <b>14</b> |
| <b>6</b> | <b>ZAKLJUČEK .....</b>                                                                            | <b>16</b> |
| <b>7</b> | <b>VIRI IN LITERATURA.....</b>                                                                    | <b>17</b> |
| <b>8</b> | <b>ZAHVALE.....</b>                                                                               | <b>18</b> |
| <b>9</b> | <b>PRILOGE .....</b>                                                                              | <b>19</b> |

## SIMBOLI, KRATICE IN ANGLEŠKE BESEDE

ER-diagram – grafični prikaz podatkovnega modela, ki prikazuje entitete (tabele), njihove attribute in relacije med njimi. Uporablja se pri načrtovanju podatkovne baze.

ID – enolična identifikacijska oznaka zapisa v podatkovni bazi. Najpogosteje predstavlja primarni ključ.

SQL-poizvedba – ukaz, zapisan v jeziku SQL, namenjen delu s podatkovno bazo (iskanje, vstavljanje, spreminjanje ali brisanje podatkov).

ŠCV – Šolski center Velenje

ERS – Elektro in računalniška šola

ERD – Entity Relationship Diagram (diagram entitet in relacij)

SQL – Structured Query Language (strukturirani poizvedovalni jezik)

JDBC – Java Database Connectivity

IDE – Integrated Development Environment (integrirano razvojno okolje)

RDBMS – Relational Database Management System (relacijski sistem za upravljanje podatkovnih baz)

PostgreSQL – odprtokodni relacijski sistem za upravljanje podatkovnih baz, namenjen zanesljivemu in varnemu shranjevanju podatkov.

DataGrip – razvojno orodje za delo s podatkovnimi bazami, ki omogoča pisanje SQL-poizvedb, urejanje podatkov in pregled strukture baze.

Supabase – platforma, ki omogoča uporabo gostovane PostgreSQL podatkovne baze brez lastne strežniške infrastrukture.

GitHub – spletna platforma za shranjevanje in upravljanje programske kode z uporabo sistema za nadzor različic Git.

GitHub Desktop – namizna aplikacija za grafično upravljanje Git repozitorijev brez uporabe ukazne vrstice.

Repository (repozitorij) – centralno mesto za shranjevanje programske kode in zgodovine sprememb.

Commit – potrjena sprememba kode v repozitoriju z opisom spremembe.

Trigger – sprožilec v podatkovni bazi, ki se samodejno izvede ob določenem dogodku (vstavljanje, posodobitev ali brisanje podatkov).

Login – prijava uporabnika v aplikacijo z uporabo poverilnic.

Root directory – osnovna oziroma izhodiščna mapa projekta.

Libraries – programske knjižnice, ki vsebujejo že pripravljene funkcije in razrede.

Environment datoteke – konfiguracijske datoteke, ki vsebujejo nastavitve okolja (npr. podatke za povezavo z bazo).

JDBC interface – standardni programski vmesnik, ki omogoča povezavo Java aplikacije s podatkovno bazo.

# 1 UVOD

Vodenje evidence zaposlenih je pomemben del poslovanja vsakega podjetja, saj omogoča pregled nad kadrovskimi podatki, delovnimi mesti, oddelki in drugimi informacijami, ki so nujne za organizacijo dela. V praksi se ti podatki pogosto vodijo v preglednicah ali ročno, kar lahko povzroča nepreglednost, napake pri vnosu ter oteženo posodabljanje in iskanje informacij. Tak način dela je neučinkovit, zlasti pri večjem številu zaposlenih, in ne omogoča sledljivosti sprememb ali enostavne centralizacije podatkov.

Namen te naloge je izdelati namizno aplikacijo za beleženje zaposlenih v podjetju, ki omogoča centralizirano in pregledno upravljanje kadrovskih podatkov. Aplikacija predstavlja izboljšavo v primerjavi s klasičnimi načini vodenja evidenc, saj omogoča strukturirano shranjevanje podatkov, lažje urejanje ter večjo zanesljivost pri delu z informacijami. Rešitev prinaša boljšo organizacijo podatkov in poenostavi vsakodnevno delo uporabnika.

Cilj naloge je razviti funkcionalno namizno aplikacijo, ki omogoča dodajanje, urejanje, brisanje in pregled zaposlenih ter povezovanje podatkov z drugimi evidencami, kot so oddelki in kraji. Pri tem je poudarek na preglednem uporabniškem vmesniku, pravilnem shranjevanju podatkov in stabilnem delovanju aplikacije.

Naloga je bila izvedena z načrtovanjem strukture rešitve, razvojem aplikacije ter testiranjem njenega delovanja. V nadaljevanju dokumentacije je podrobneje predstavljeno uporabljeno delovno okolje, potek izdelave aplikacije in analiza rezultatov.

## 2 PREGLED STANJA TEHNIKE

### 2.1 PostgreSQL

PostgreSQL je odprtokodni relacijski sistem za upravljanje podatkovnih baz, ki se pogosto uporablja v poslovnih in industrijskih okoljih. Omogoča zanesljivo shranjevanje podatkov, delo z relacijami med tabelami ter uporabo naprednih mehanizmov, kot so transakcije, omejitve in funkcije. Zaradi stabilnosti, razširljivosti in skladnosti s standardi SQL je PostgreSQL primeren za aplikacije, kjer je pomembna varnost in celovitost podatkov.<sup>1</sup>



*Slika 1: PostgreSQL logo*

### 2.2 DataGrip

DataGrip je razvojno orodje podjetja JetBrains, namenjeno delu s podatkovnimi bazami. Omogoča povezovanje z različnimi sistemi za upravljanje podatkovnih baz, pregledovanje struktur tabel, izvajanje SQL poizvedb ter upravljanje podatkov. Orodje omogoča pregledno delo z bazo in olajša razvoj ter testiranje aplikacij, ki uporabljajo podatkovno bazo.<sup>2</sup>



*Slika 2: DataGrip logo*

---

<sup>1</sup> PostgreSQL Global Development Group, **PostgreSQL 17 Documentation** <https://www.postgresql.org/>

<sup>2</sup> JetBrains, **DataGrip documentation** <https://www.jetbrains.com/datagrip/>

## 2.3 Supabase

Supabase je sodobna platforma, ki ponuja gostovano okolje za PostgreSQL podatkovno bazo ter dodatne storitve za razvoj aplikacij. Omogoča uporabo PostgreSQL baze brez potrebe po lastni strežniški infrastrukturi, kar poenostavi razvoj in testiranje aplikacij. Zaradi dostopnosti in podpore za standardne povezave je Supabase primeren za razvojne in manjše produkcijske projekte.<sup>3</sup>



*Slika 3: Supabase logo*

## 2.4 GitHub

GitHub je spletna platforma za gostovanje in upravljanje programske kode, ki temelji na sistemu za nadzor različic Git. Omogoča shranjevanje izvorne kode v repozitorijih, spremljanje sprememb ter sodelovanje pri razvoju programske opreme. Z uporabo GitHuba je mogoče slediti razvoju projekta, pregledovati zgodovino sprememb in vzdrževati urejeno strukturo kode.

Pri izdelavi aplikacije je GitHub uporabljen kot osrednje mesto za shranjevanje programske kode projekta. Tak način dela omogoča boljšo organizacijo razvoja, enostavno varnostno kopiranje kode ter preglednost nad razvojnimi fazami projekta. Poleg tega objava kode na GitHubu predstavlja dobro prakso in omogoča vpogled v rešitev tudi drugim uporabnikom ali ocenjevalcem.<sup>4</sup>



*Slika 4: GitHub logo*

---

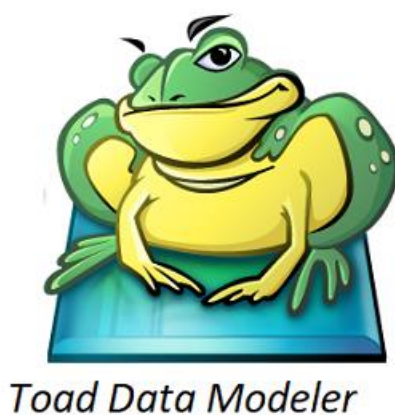
<sup>3</sup> Supabase, **Database connection management / pooling (Session Pooler, Direct connection):** <https://supabase.com/>

<sup>4</sup> GitHub Docs, **About repositories:** <https://github.com/>

## 2.5 Toad Data Modeler

Toad Data Modeler je orodje za načrtovanje in modeliranje podatkovnih baz, ki omogoča grafični prikaz strukture podatkov, tabel in relacij med njimi. Z uporabo ER-diagramov je mogoče pregledno prikazati logično zasnovo podatkovne baze še pred samo implementacijo. Tak pristop zmanjšuje možnost napak pri zasnovi, olajša razume<sup>5</sup>vanje strukture baze ter omogoča enostavnejše prilagoditve v zgodnjih fazah razvoja.

Pri izdelavi aplikacije je bil Toad Data Modeler uporabljen za načrtovanje relacijske strukture podatkovne baze, kar je omogočilo jasen pregled nad povezavami med tabelami, primarnimi in tujimi ključi ter osnovnimi omejitvami.



*Slika 5: Toad Data Modeler logo*

---

<sup>5</sup> Toad Data Modeler – Support/Documentation: [https://en.wikipedia.org/wiki/Toad\\_Data\\_Modeler](https://en.wikipedia.org/wiki/Toad_Data_Modeler)

## 2.6 IntelliJ IDEA

IntelliJ IDEA je razvojno okolje za programski jezik Java in druge programske jezike, ki omogoča celovit razvoj aplikacij v enem okolju. Orodje ponuja podporo za pisanje, urejanje in razhroščevanje kode, upravljanje projektne strukture ter integracijo z zunanjimi orodji, kot so sistemi za nadzor različic in podatkovne baze. Zaradi preglednega uporabniškega vmesnika in naprednih funkcij je IntelliJ IDEA pogosto uporabljen pri razvoju namiznih in spletnih aplikacij.<sup>6</sup>

Pri izdelavi namizne aplikacije je bil IntelliJ IDEA uporabljen kot glavno razvojno okolje. Omogočal je učinkovito organizacijo projektnih datotek, lažje odpravljanje napak ter sprotno testiranje delovanja aplikacije. Integracija z orodji za delo z GitHubom in podpora za delo z Java projekti sta prispevali k bolj preglednemu in strukturiranemu razvoju aplikacije.



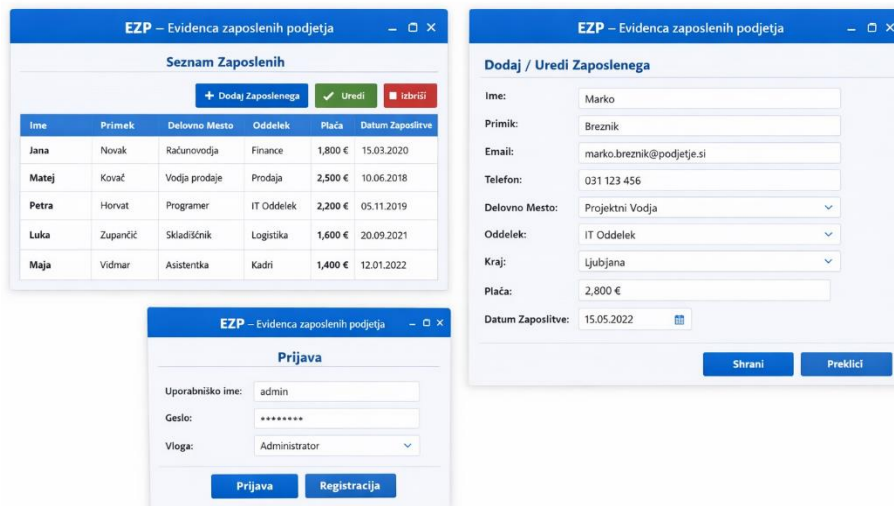
*Slika 6: Logo IntelliJ IDEA*

---

<sup>6</sup> JetBrains, **IntelliJ IDEA Documentation**: <https://www.jetbrains.com/help/idea/installation-guide.html>

### 3 METODE DELA

V tem poglavju so predstavljeni postopki in metode, uporabljeni pri razvoju namizne aplikacije. Opisan je zaporedni potek dela, od priprave razvojnega okolja do testiranja in končne povezave z gostovano podatkovno bazo.



Slika 7: Predhodnji izgled aplikacije

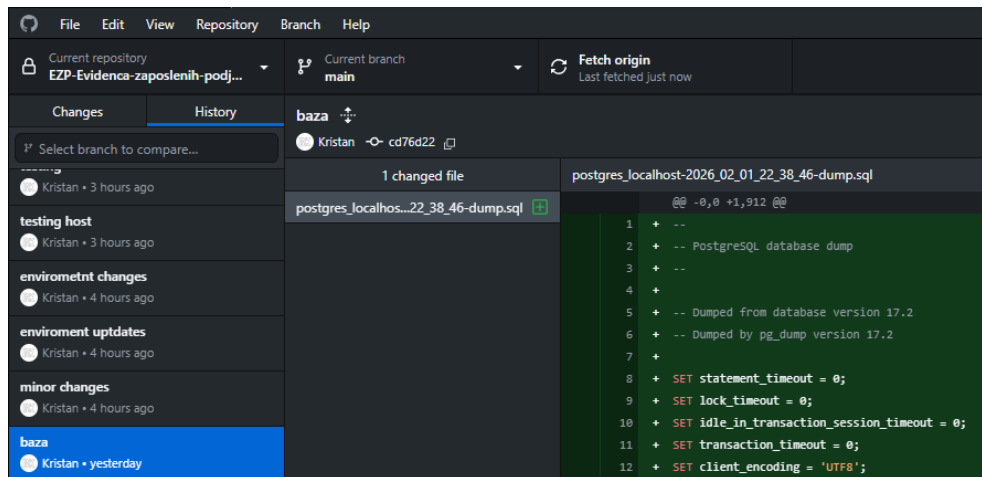
#### 3.1 Priprava GitHub repozitorija

Na začetku projekta je bil ustvarjen repozitorij na platformi GitHub, ki je služil kot osrednje mesto za shranjevanje in upravljanje programske kode. Repozitorij je omogočal nadzor nad različicami kode, pregled zgodovine sprememb ter varnostno kopiranje projekta. Tak način dela zagotavlja preglednost razvoja in olajša morebitne nadaljnje nadgradnje aplikacije.

##### 3.1.1 Priprava Github desktop

Za lažje upravljanje repozitorija in delo z različicami kode je bilo uporabljeno namizno orodje GitHub Desktop. Orodje omogoča grafični pregled nad spremembami v projektu, enostavno ustvarjanje potrjenih sprememb (commitov) ter pošiljanje kode v oddaljeni repozitorij brez uporabe ukazne vrstice. S tem je bilo delo z nadzorom različic preglednejše in manj dovzetno za napake.

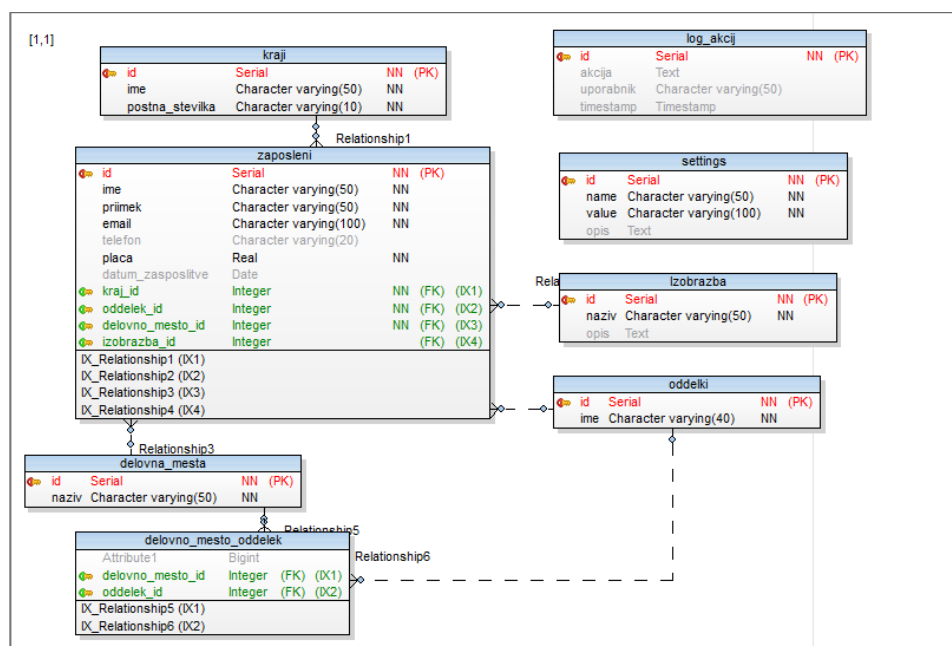
GitHub Desktop je bil povezan z ustvarjenim repozitorijem na platformi GitHub, kar je omogočilo sinhronizacijo lokalnega projekta z oddaljenim repozitorijem. Med razvojem aplikacije so bile spremembe redno shranjevalne in poslane v repozitorij, kar je zagotovilo sledljivost razvoja in varnostno kopiranje programske kode.



Slika 8: Izgled delovanja Github desktop

### 3.2 Načrtovanje podatkovnega modela (ER-diagram)

Na podlagi zahtev aplikacije je bil najprej izdelan ER-diagram, ki prikazuje entitete, njihove attribute ter relacije med njimi. Poseben poudarek je bil namenjen pravilni uporabi primarnih in tujih ključev ter normalizaciji podatkov, kar pripomore k večji celovitosti in doslednosti podatkovne baze. ER-diagram je služil kot osnova za kasnejšo implementacijo podatkovne baze.

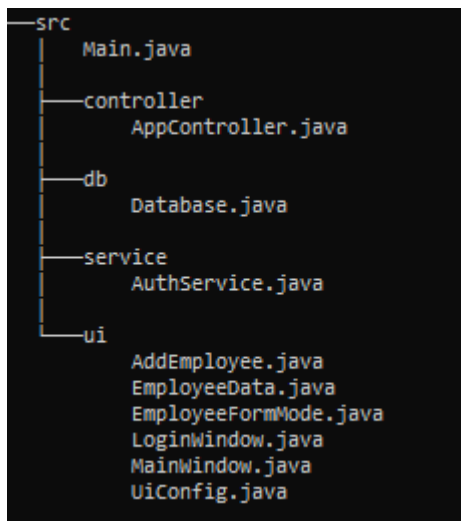


Slika 9: Načrt ER modela

### 3.3 IZDELAVA APLIKACIJE V INTELJ

#### 3.3.1 STROKTURA PROJEKTA

Glavni del kode se je odvijal v poglavju src ki je bil root directory poleg tega so bili tudi še Libraries ter enviroment datoteke



Slika 10: stroktura projekta

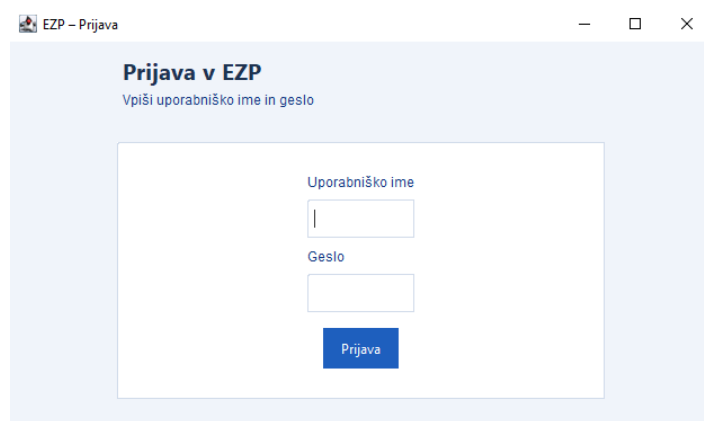
#### 3.3.2 POTEK IZDELAVE PROJEKTA

Izdelava aplikacije v razvojnem okolju IntelliJ IDEA je potekala postopno in sistematično, z namenom sprotnega preverjanja delovanja ter lažjega odpravljanja napak. Najprej sem si zastavil osnovne zahteve aplikacije, kot so pregled seznama zaposlenih, dodajanje novih zaposlenih, urejanje obstoječih podatkov in brisanje zapisov. Poseben poudarek je bil na preglednosti aplikacije in pravilnem delovanju posameznih funkcionalnosti.

V naslednjem koraku sem v IntelliJ IDEA ustvaril osnovno strukturo projekta. Projekt sem razdelil na več logičnih delov, kot so razredi za povezavo s podatkovno bazo, razredi za obdelavo podatkov ter razredi, namenjeni uporabniškemu vmesniku. Takšna razdelitev je omogočila boljšo preglednost kode in lažje nadaljnje nadgrajevanje aplikacije.

Ko je bila osnovna struktura projekta pripravljena, sem najprej preveril delovanje povezave do podatkovne baze ter izvajanje osnovnih SQL-poizvedb. Ta korak je bil pomemben, saj je omogočil hitro zaznavanje morebitnih napak pri komunikaciji z bazo in pri obdelavi podatkov.

Po uspešnem delovanju povezave sem začel z razvojem uporabniškega vmesnika. Najprej sem izdelal glavno okno aplikacije in tabelo za prikaz zaposlenih. Nato sem dodal obrazce za dodajanje in urejanje podatkov zaposlenih. Pri tem sem poskrbel tudi za osnovno preverjanje vnosov, kot so obvezna polja in pravilnost vnesenih podatkov, da bi zmanjšal možnost napačnih vnosov.



Slika 11: Login okno

Med razvojem sem aplikacijo večkrat zagnal in sproti preverjal delovanje posameznih delov kode. Takšen način dela mi je omogočil, da sem napake odkril že v zgodnjih fazah razvoja in jih učinkovito odpravil. Razvoj v IntelliJ IDEA je tako potekal postopno, z jasnim poudarkom na stabilnosti, preglednosti kode in pravilnem delovanju aplikacije.

### 3.4 Začetno testiranje na lokalnem okolju z DataGripom

Pred uporabo gostovane podatkovne baze je bilo testiranje izvedeno v lokalnem okolju. S pomočjo orodja DataGrip so bile ustvarjene tabele, preverjene relacije ter testirane osnovne SQL poizvedbe. Lokalno testiranje je omogočilo hitrejše odpravljanje napak in preverjanje pravilnosti zasnove brez vpliva na produkcijsko okolje.

### 3.5 Vzpostavitev povezave na Supabase

Po uspešno zaključenem lokalnem testiranju je bila aplikacija povezana z gostovano PostgreSQL podatkovno bazo na platformi Supabase. Povezava je bila vzpostavljena preko JDBC vmesnika, kar omogoča standardizirano in varno komunikacijo med aplikacijo in podatkovno bazo. S tem je bila zagotovljena dostopnost podatkov tudi izven lokalnega okolja.

### **3.6 Testiranje delovanja aplikacije**

V zaključni fazi so bila izvedena različna testiranja delovanja aplikacije. Preverjeno je bilo dodajanje, urejanje in brisanje zaposlenih ter pravilno shranjevanje podatkov v podatkovni bazi. Poleg tega je bilo preverjeno tudi obnašanje aplikacije ob napačnih vnosih in napakah pri povezavi z bazo, kar prispeva k večji zanesljivosti končne rešitve.

## 4 PROŽILCI

Pri zasnovi podatkovne baze sem uporabil sprožilce (triggerje), ki omogočajo samodejno izvajanje določenih postopkov ob spremembah podatkov. Namen uporabe sprožilcev je povečati zanesljivost podatkov, zmanjšati možnost napak ter zagotoviti boljši nadzor nad spremembami v bazi. Sprožilci delujejo neodvisno od aplikacije, kar pomeni, da se pravila izvajajo vedno, ne glede na način dostopa do podatkov.

### 4.1 Prožilec 1

#### Sprožilec za beleženje časa zadnje spremembe zapisa - trg\_log\_zaposleni

Ta trigger je namenjen beleženju sprememb nad podatki zaposlenih. Ob vsaki spremembi zapisa (npr. posodobitev podatkov o zaposlenem) se v posebno dnevniško tabelo samodejno zapiše informacija o spremembi.

Namen tega triggerja je sledljivost sprememb, saj omogoča pregled nad tem, kateri podatki so bili spremenjeni in kdaj. Takšen pristop je pomemben z vidika nadzora nad podatki ter lažjega odkrivanja napak ali nepooblaščenih sprememb. Definitioon temelji na klici podprograma 'log\_zaposleni\_changes'

```
BEGIN
  INSERT INTO log_akcij(akcija, uporabnik, "timestamp")
  VALUES (
    TG_OP || ' zaposleni id=' || COALESCE(NEW.id, OLD.id),
    current_user,
    now()
  );

  IF TG_OP = 'DELETE' THEN
    RETURN OLD;
  END IF;

  RETURN NEW;
END;
```

Slika 12: Definicija prvega prožilca

## 4.2 Prožilec 2

### Sprožilec za preprečevanje neveljavnih vnosov - `fix_negative_placa`

Ta trigger skrbi za pravilnost vnosa podatkov o plači zaposlenega. Ob vstavljanju ali posodabljanju zapisa preveri, ali je vrednost plače negativna.

Če je zaznana neveljavna (negativna) vrednost, trigger popravi ali zavrne tak vnos, s čimer prepreči shranjevanje nelogičnih podatkov v bazo. Namen tega triggerja je zagotavljanje podatkovne pravilnosti in preprečevanje napak, ki bi lahko nastale zaradi napačnega vnosa uporabnika.

```
BEGIN
  IF NEW.placa < 0 THEN
    -- nastavi na 1000
    NEW.placa := 1000;

    -- zapiši v log
    INSERT INTO log_akcij(akcija, uporabnik, "timestamp")
    VALUES (
      'Vnos negativne plače. Vrednost je bila popravljena na 1000
      (zaposleni-ID=' || NEW.id || ')'
      , current_user,
      now()
    );
  END IF;

  RETURN NEW;
END;
```

Slika 13: Definicija delovanja aplikacije

## 4.3 Prožilec 3

### Sprožilec za samodejno obdelavo brisanja povezanih podatkov - fix\_future\_datum\_zaposlitve

Ta trigger je uporabljen za nadzor nad datumom zaposlitve. Preverja, ali je vneseni datum zaposlitve postavljen v prihodnost, kar v praksi ni smiselno.

Če je zaznan nepravilen datum, trigger samodejno popravi vrednost ali prepreči shranjevanje zapisa. Namen tega triggerja je zagotoviti časovno pravilne podatke ter preprečiti napake pri vnosu datumov.

```
BEGIN
  IF NEW.datum_zaposlitve > CURRENT_DATE THEN
    NEW.datum_zaposlitve := CURRENT_DATE;

    INSERT INTO log_akcij(akcija, uporabnik, "timestamp")
    VALUES ('POPRAVLJEN DATUM ZAPOSLOTVE (AUTO)', current_user, now());
  END IF;

  RETURN NEW;
END;
```

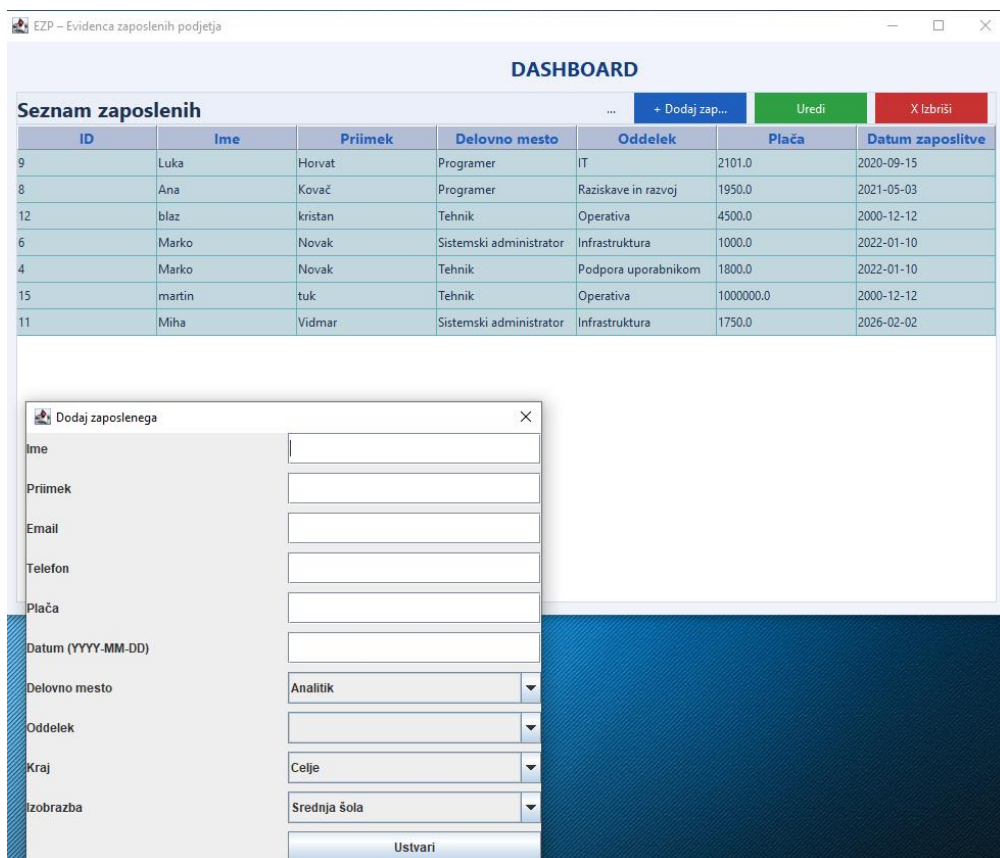
Slika 14: Definicija delovanja prožilca 3

## 5 REZULTATI

V tem poglavju so predstavljeni doseženi rezultati izdelave namizne aplikacije za beleženje zaposlenih v podjetju. Na podlagi zastavljenih ciljev v uvodu je bilo preverjeno, ali aplikacija izpolnjuje funkcionalne in tehnične zahteve ter ali omogoča zanesljivo delo z zaposlenimi podatki.

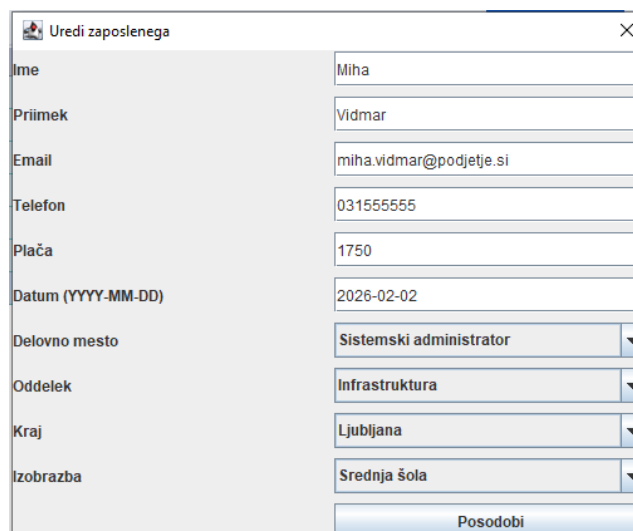
Rezultat projekta je delujoča namizna aplikacija, ki omogoča centralizirano upravljanje podatkov o zaposlenih. Uporabnik lahko v aplikaciji dodaja nove zaposlene, ureja obstoječe zapise, briše zaposlene ter pregleduje podatke v pregledni tabeli. Vsi podatki se shranjujejo v relacijsko podatkovno bazo PostgreSQL, kar zagotavlja večjo zanesljivost in trajnost podatkov v primerjavi z ročnim ali datotečnim vodenjem evidenc.

Pomemben rezultat je tudi pravilna zasnova podatkovne baze. Z uporabo ER-diagrama in relacij med tabelami so podatki logično povezani, kar zmanjšuje podvajanje in omogoča doslednost podatkov. Uporaba sprožilcev (triggerjev) je dodatno izboljšala delovanje sistema, saj se samodejno preverjajo in popravljajo določeni vnosi (npr. negativne plače ali neveljavni datumi zaposlitve) ter beležijo spremembe v ločeno dnevniško tabelo. S tem je zagotovljena večja podatkovna integriteta in sledljivost sprememb.



Slika 15: Predogled končane aplikacije

Aplikacija je bila uspešno preizkušena tako v lokalnem okolju kot tudi ob povezavi na gostovano podatkovno bazo Supabase. Testiranja so pokazala, da aplikacija stabilno deluje pri običajni uporabi, pravilno obravnava napačne vnose in se ustrezno odzove na težave s povezavo do baze.



|                           |                          |
|---------------------------|--------------------------|
| Ime                       | Miha                     |
| Priimek                   | Vidmar                   |
| Email                     | miha.vidmar@podjetje.si  |
| Telefon                   | 031555555                |
| Plača                     | 1750                     |
| Datum (YYYY-MM-DD)        | 2026-02-02               |
| Delovno mesto             | Sistemiški administrator |
| Oddelek                   | Infrastruktura           |
| Kraj                      | Ljubljana                |
| Izobrazba                 | Srednja šola             |
| <button>Posodobi</button> |                          |

*Slika 16: 2 del. Predogled končane aplikacije*

Končni rezultat projekta je funkcionalna, pregledna in razširljiva namizna aplikacija, ki predstavlja ustrezen rešitev za beleženje zaposlenih v manjšem podjetju ali organizaciji. Rešitev izpolnjuje zastavljene cilje naloge in hkrati ponuja dobro osnovo za morebitne nadaljnje nadgradnje.

## 6 ZAKLJUČEK

V zaključni nalogi je bil obravnavan problem nepreglednega in neučinkovitega vodenja evidence zaposlenih v podjetju. Namen naloge je bil razviti namizno aplikacijo, ki omogoča centralizirano, pregledno in zanesljivo upravljanje kadrovskih podatkov. Zastavljeni cilji so bili v celoti doseženi, saj razvita aplikacija omogoča osnovne kadrovske funkcionalnosti, kot so dodajanje, urejanje, brisanje in pregled zaposlenih.

Pri razvoju aplikacije so bile uporabljene ustrezne metode dela, od načrtovanja podatkovnega modela do postopne implementacije in testiranja. Poseben poudarek je bil namenjen pravilnemu shranjevanju podatkov v relacijski podatkovni bazi PostgreSQL ter zagotavljanju podatkovne celovitosti s pomočjo triggerjev. Ti so preprečevali napačne vnose (npr. negativne plače ali napačne datume zaposlitve) in omogočali beleženje sprememb v evidenci.

Rezultati dela potrjujejo, da je izbrana rešitev primerna za praktično uporabo in predstavlja izboljšavo v primerjavi z ročnim ali tabelarnim vodenjem podatkov. Aplikacija je stabilna, pregledna in enostavna za uporabo. Možnosti za nadaljnje delo vključujejo razširitev aplikacije z uporabniškimi pravicami, naprednejšim iskanjem, statističnimi pregledi ter dodatnimi varnostnimi mehanizmi.

## 7 VIRI IN LITERATURA

- [1] PostgreSQL Global Development Group, **PostgreSQL 17 Documentation** (online), dostop: 3. 2. 2026, <https://www.postgresql.org/docs/>
- [2] JetBrains, **DataGrip documentation** (online), dostop: 3. 2. 2026, <https://www.jetbrains.com/help/datagrip/>
- [3] Supabase, **Database connection management / pooling (Session Pooler, Direct connection)** (online), dostop: 3. 2. 2026, <https://supabase.com/docs/guides/database/connecting-to-postgres>
- [4] Supabase, **Migrations (upravljanje sprememb baze)** (online), dostop: 3. 2. 2026, <https://supabase.com/docs/guides/cli/managing-migrations>
- [5] GitHub Docs, **About repositories** (online), dostop: 3. 2. 2026, <https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories>
- [6] GitHub Docs, **Creating your first repository using GitHub Desktop** (online), dostop: 3. 2. 2026, <https://docs.github.com/en/desktop/overview/creating-your-first-repository-using-github-desktop>
- [7] JetBrains, **IntelliJ IDEA Documentation (Help / tutorials / UI)** (online), dostop: 3. 2. 2026, <https://www.jetbrains.com/help/idea/>
- [8] Quest Software, **Toad Data Modeler – Support/Documentation** (online), dostop: 3. 2. 2026, <https://support.quest.com/toad-data-modeler>
- [9] pgJDBC, **PostgreSQL JDBC Driver (dokumentacija in uporaba)** (online), dostop: 3. 2. 2026, <https://jdbc.postgresql.org/>
- [10] ChatGPT (model **GPT-5.2 Thinking**), OpenAI, generirano: 3. 2. 2026, <https://chatgpt.com/>

## **8 ZAHVALE**

### **ZAHVALA**

Zahvaljujem se mentorju Islamu Mušiću  
za pomoč in vodenje pri opravljanju naloge.

Posebna zahvala velja staršem, ki mi omogočajo  
šolanje na ŠC Velenje – ERŠ-u.

## 9 PRILOGE

Priloga 1

Na konec naloge naj kandidat vključi izjavo, ki potrjuje, da je delo izdelal samostojno, pod vodstvom mentorja na šoli ali v podjetju ter po virih, ki so navedeni v bibliografiji naloge.

### Izjava avtorja naloge

Podpisan(a) \_\_\_\_\_ Blaž Kristan \_\_\_\_\_, maturant(ka) Elektro in računalniške šole Šolskega centra Velenje, programa \_\_\_\_\_ Tehnika Računalništva \_\_\_\_\_, izjavljam, da sem nalogo pripravil(a) samostojno pod vodstvom mentorja na šoli ali v podjetju ter po virih, ki so navedeni v bibliografiji naloge.

Velenje, dne \_\_\_\_\_ 29.1.2025 \_\_\_\_\_

Podpis: \_\_\_\_\_ Kristan \_\_\_\_\_